✅ 1. Enterprise Systems

Enterprise systems are **large-scale applications** designed to support **business operations** of an organization.

Key Features

- Serve **business users** (sales, HR, finance, operations, etc.)
- Handle **complex workflows**
- Focus on **scalability, security, reliability**
- Usually consist of multiple modules like **ERP, CRM, HRMS, SCM**

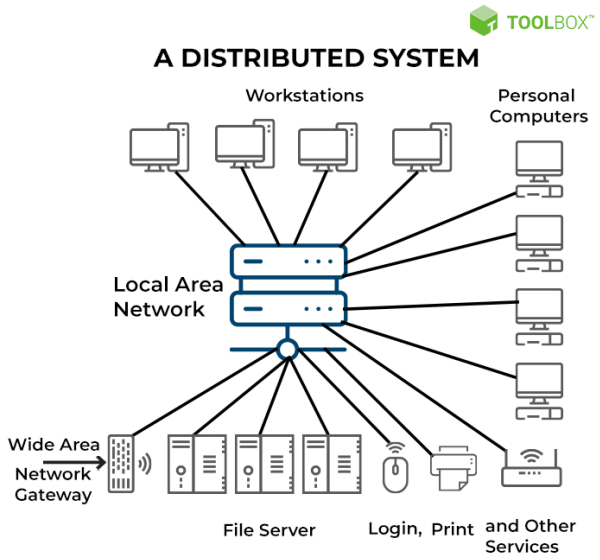
Examples

- ERP systems (SAP, Oracle ERP)
- Banking core systems
- Large e-commerce platforms (Amazon backend)

Simple Definition

➡️ **An enterprise system supports big business operations and processes.**

✅ 2. Distributed Systems



A distributed system is a system where **multiple computers (nodes) work together**, but they appear as **one system** to the user.

Key Features

- Runs across multiple **machines/nodes**
- Nodes communicate over **network**
- Focus on **scalability, fault tolerance, replication**
- Data and processing are **distributed**

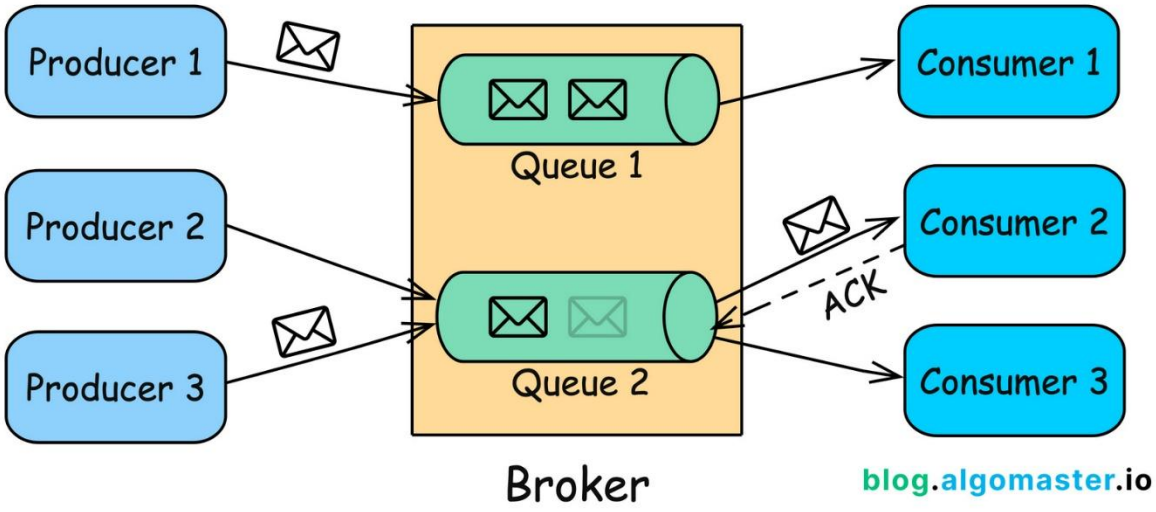
Examples

- Microservices architecture
- Distributed databases (Cassandra, MongoDB cluster)
- Cloud systems (AWS, Azure)
- Kafka clusters

Simple Definition

➡️ **A distributed system spreads work across multiple machines.**

✅ 3. Asynchronous Systems



An asynchronous system allows components to **send and receive messages without waiting** for each other.

Key Features

- Non-blocking operations
- Message-based communication
- Improves responsiveness and performance
- Enables loose coupling
- Often uses queues or event streams

Common Technologies

- Message queues: RabbitMQ, Azure Service Bus
- Event streaming: Kafka
- Async programming patterns (async/await, callbacks)

Simple Definition

➡️ **An asynchronous system does not wait — it continues working and processes tasks later.**

Concept	Key Idea	Example
Enterprise System	Big business system	A bank's complete backend
Distributed System	Runs on many machines	Microservices on Kubernetes
Async System	Does not wait; uses messaging	Order service → publish "OrderCreated" event

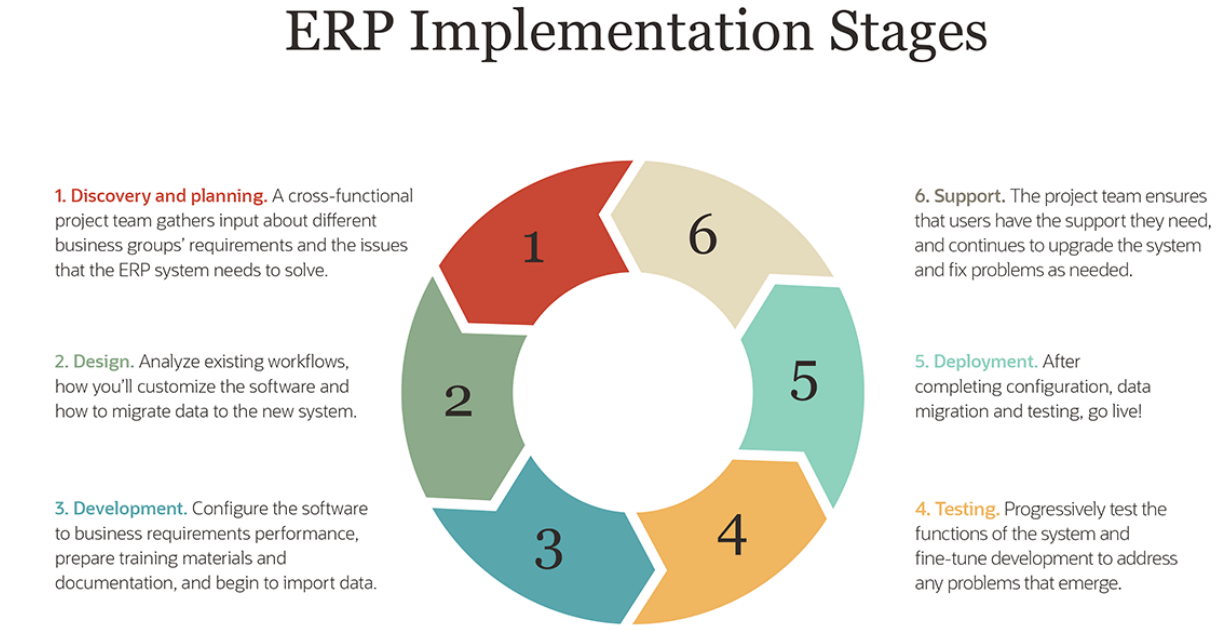
🎯 How They Relate

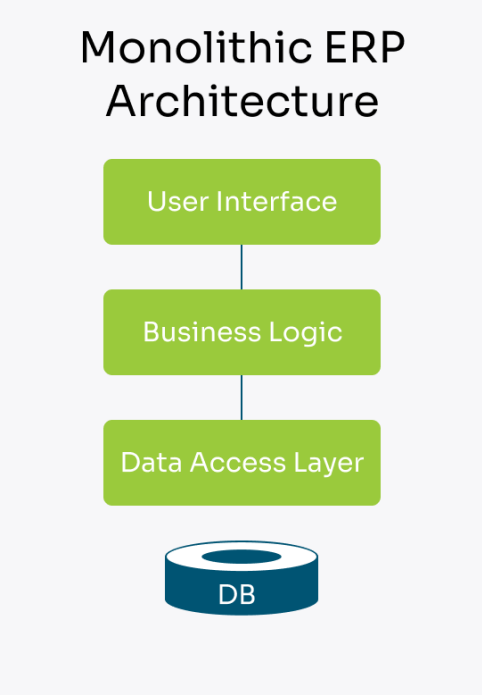
Many modern systems are **all three**:

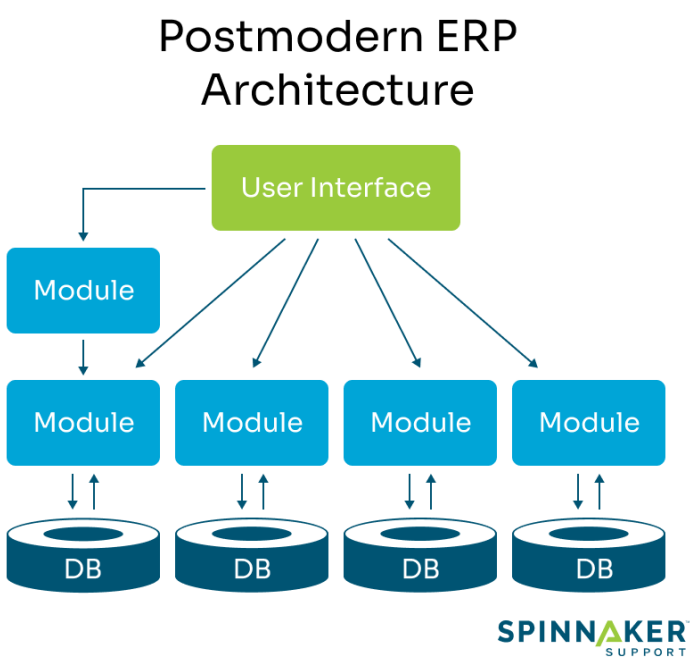
Example: E-commerce Order System

Layer	How it fits
Enterprise	Whole system handles customers, orders, payments, shipping
Distributed	Microservices: Order, Inventory, Payment, Shipping
Asynchronous	Order service publishes events, other services react later

✅ 1. Enterprise Systems — Use Cases







4

Enterprise systems support **large business operations** with end-to-end workflows.

Use Case 1: Bank Core System

What happens:

- Customer account management
- Loan processing
- KYC validation
- Transaction history
- Compliance rules

Why this is an Enterprise System:

- It supports major **business processes** across the entire organization.
- Many departments depend on it: accounts, compliance, audit, branch operations.
- Requires strong **security, reliability, and consistency**.

Use Case 2: ERP System (SAP, Oracle) in a Manufacturing Company

What it handles:

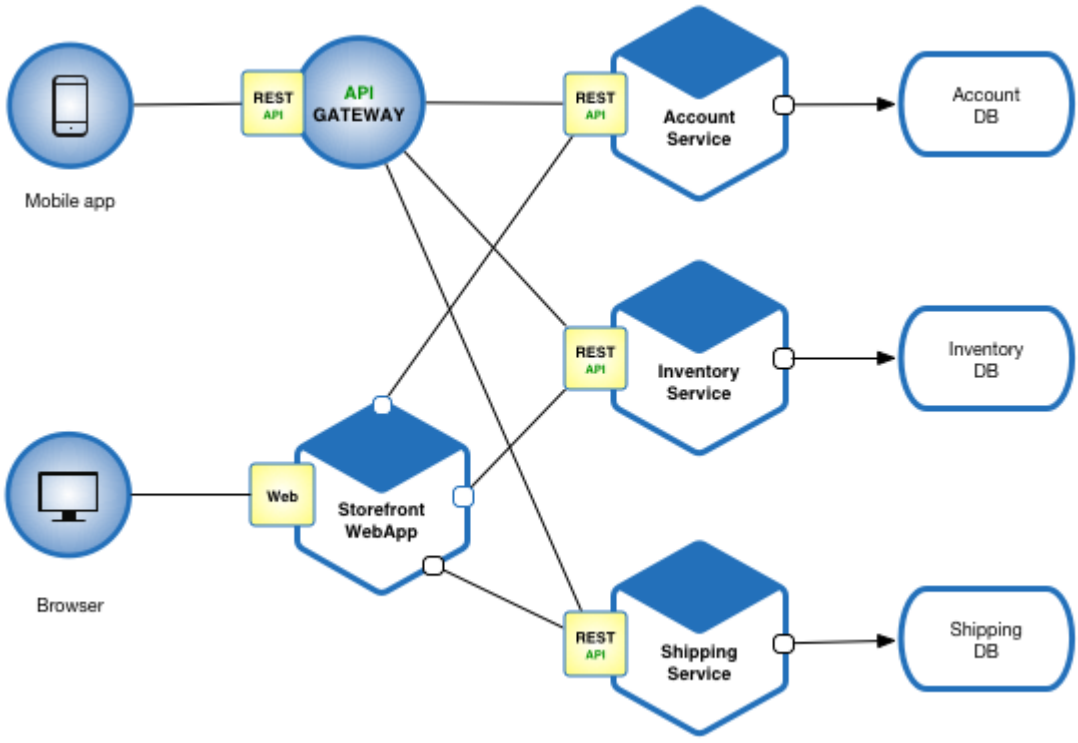
- Inventory
- Procurement
- Order management
- HR payroll
- Finance & reporting

Why this is Enterprise:

- Everything is tied to business workflows.
- Designed to be used by thousands of employees inside the company.

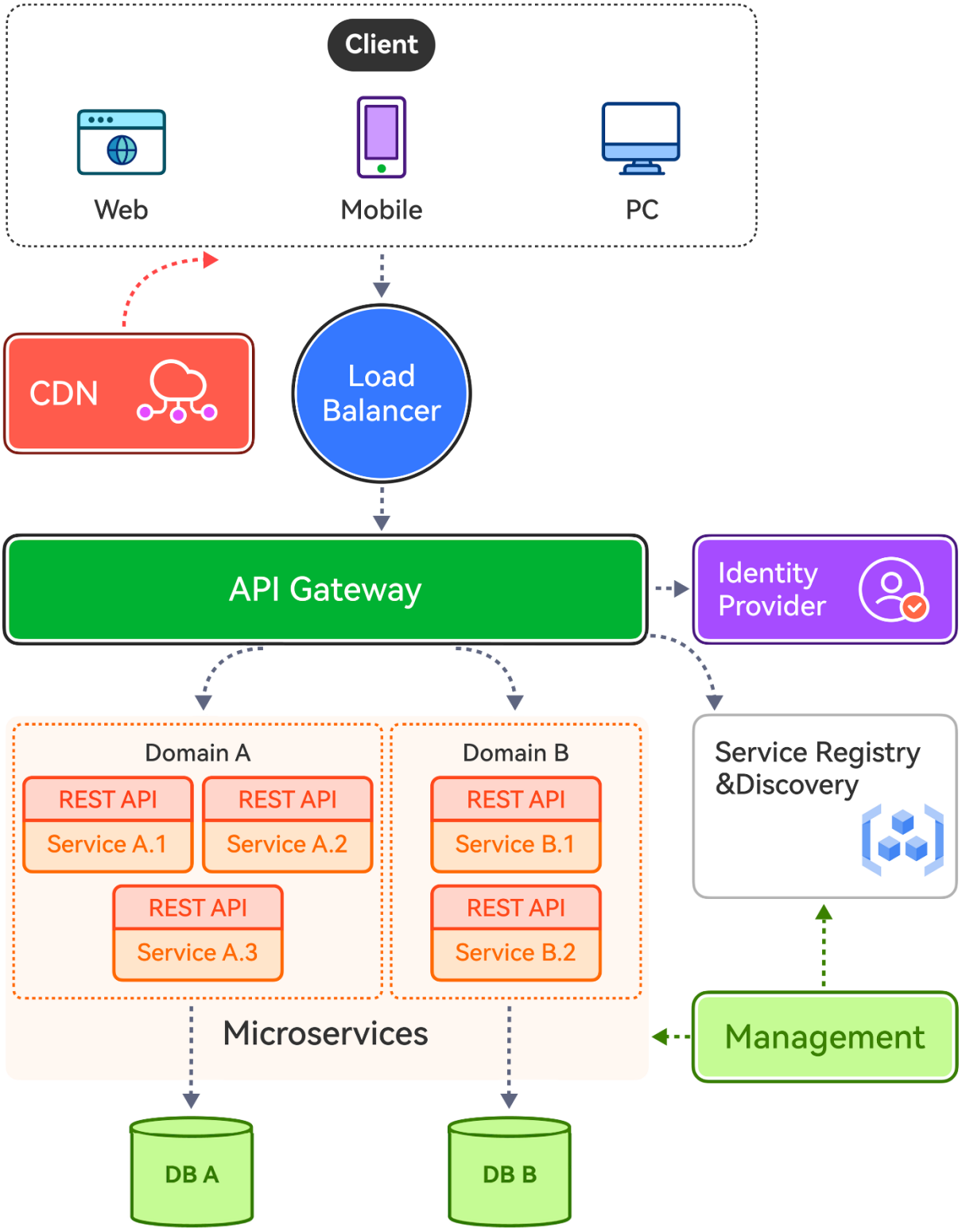
➡️ **Enterprise = Business-level solutions for the entire organization.**

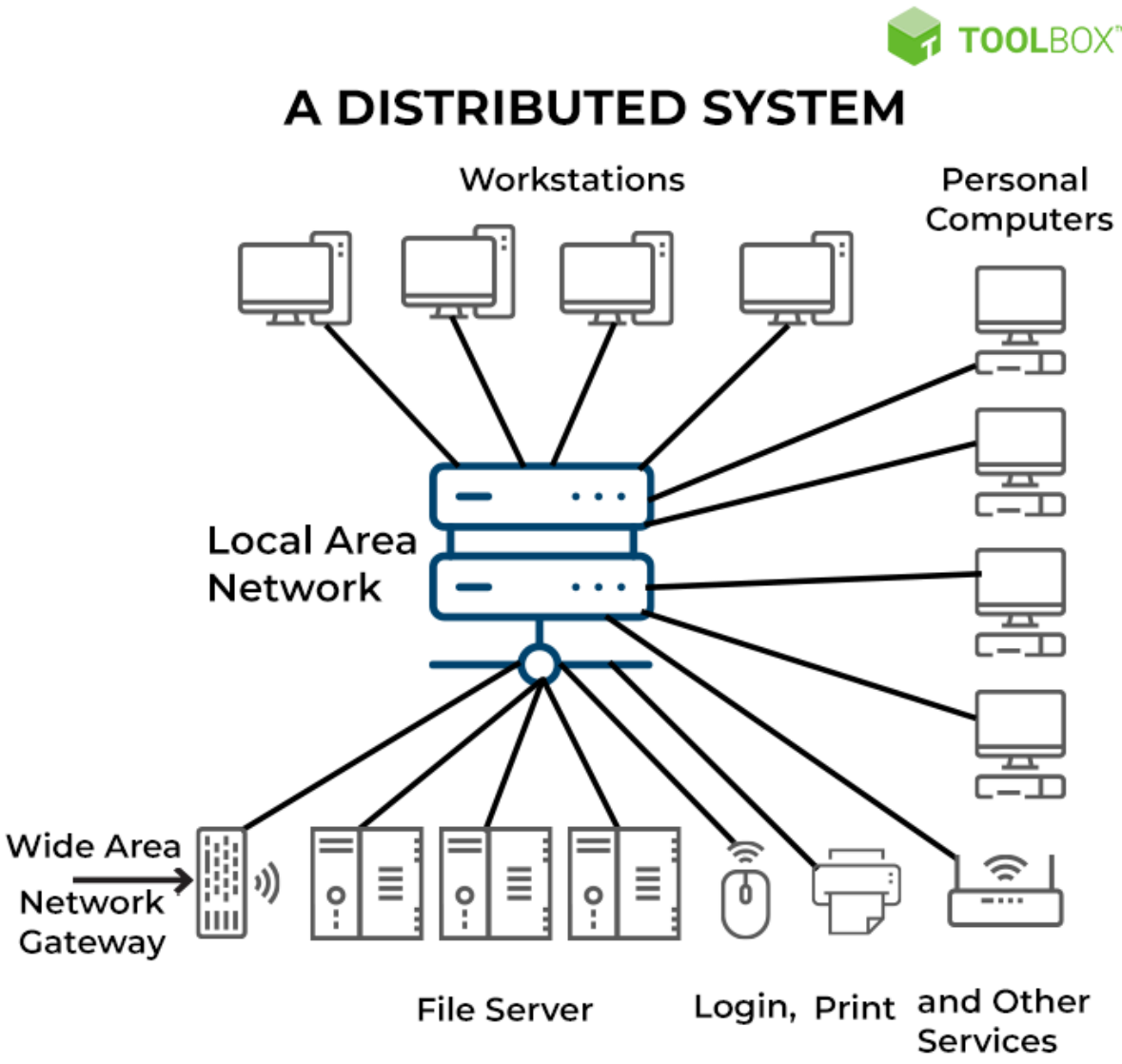
✅ **2. Distributed Systems — Use Cases**



Typical Microservice Architecture

ByteByteGo





4

Distributed systems run on **multiple machines** that work together.

Use Case 1: Netflix Streaming

How it works:

- Video stored on servers worldwide (CDN)
- When you watch a movie, the nearest server streams it

- Recommendation service runs on separate machines
- User authentication runs separately
- Billing service runs separately

Why this is a Distributed System:

- Workload is split across **hundreds of servers**
- Services talk to each other via APIs
- If one service fails, others continue
- Scales globally

Use Case 2: E-commerce Microservices (Amazon Example)

Architecture:

- **Order Service**
- **Inventory Service**
- **Payment Service**
- **Shipping Service**
- **Notification Service**

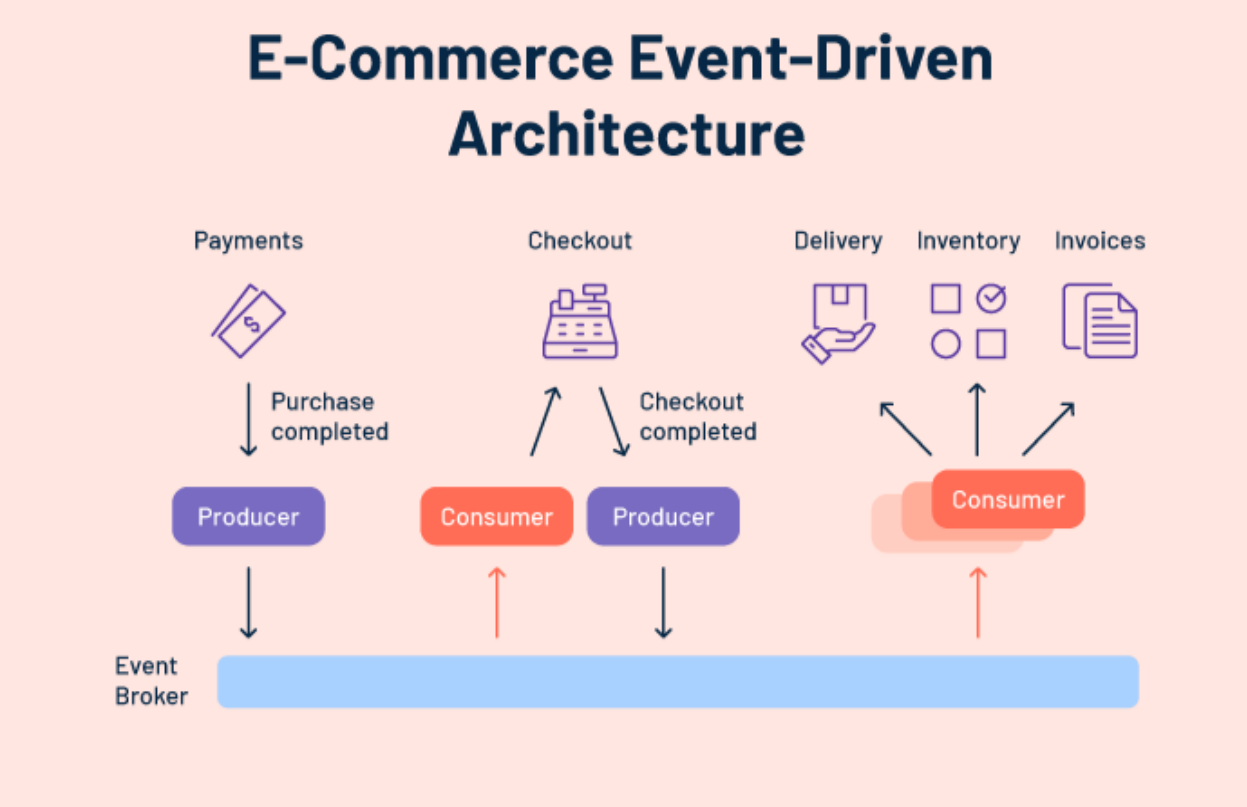
Each service runs on **different servers or containers**.

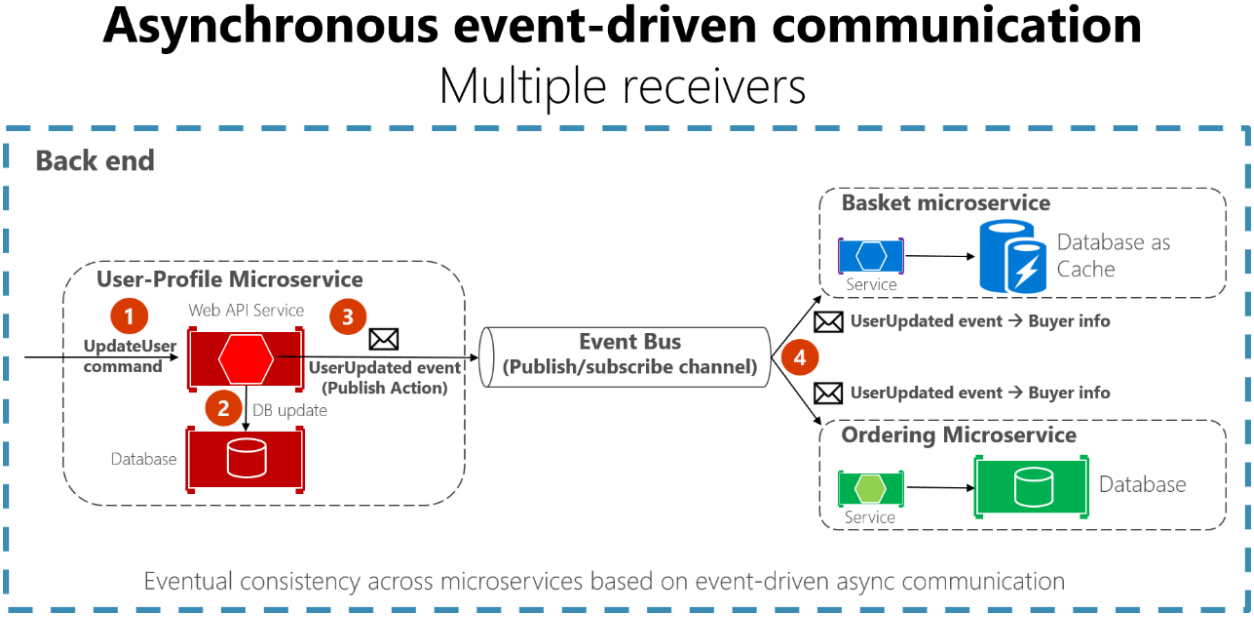
Why Distributed:

- Lower coupling
- Easy scaling
- Independent deployment
- High reliability

➡️ **Distributed = System is spread across multiple machines and works as one.**

✅ **3. Asynchronous Systems — Use Cases**





4

Asynchronous systems allow tasks to run **without waiting**, usually using events or queues.

Use Case 1: Placing an Order on an E-commerce App

Step-by-step:

1. User clicks **Place Order**
2. Order service **immediately returns success**
3. Behind the scenes:
    - Publishes an event: *OrderCreated*
    - Inventory service listens → reduces stock
    - Payment service listens → charges card
    - Notification service listens → sends SMS/email

Why this is Asynchronous:

- The order API **does not wait** for payment, inventory update, email sending
- Everything happens **in background** using events

Use Case 2: Uploading a File on Google Drive

What user sees:

- Upload starts → UI remains responsive
- Thumbnails appear later

Internally:

- File chunks are uploaded
- Another service processes thumbnails
- Another service extracts metadata

Why Asynchronous:

- Operations are **long-running**
- System doesn't block
- User doesn't wait for completion

Use Case 3: Sending OTP SMS

Process:

- Your app sends OTP → **API responds instantly**
- SMS is queued in background
- SMS gateway picks it and sends it

Why Async:

- Avoids delay
- Prevents request timeout

➡️ **Asynchronous = Do not wait; continue and process in background.**

🎯 **How All Three Combine in Real Life**

Example: Amazon Order System

Layer	**Explanation**
Enterprise	Entire ecommerce platform supporting business (orders, payments, returns, logistics)
Distributed	Microservices running on many servers across regions
Asynchronous	Order events trigger payment, inventory, notification asynchronously